

DOCUMENTAÇÃO TÉCNICA

Plataforma de Aprendizado DevAcademy

Arquitetura de Sistemas Web, Fundamentação Teórica e
Segurança de Dados em Camadas

Engenharia de Software & Desenvolvimento Web
Ambiente Integrado: **HTML5, CSS3, PHP 8 e MySQL**

Documentação Técnica: Plataforma DevAcademy

Análise arquitetural, isolamento de escopo, persistência de dados relacionais e interatividades dinâmicas assíncronas.

O presente documento detalha a concepção, arquitetura e implementação da plataforma LMS DevAcademy, englobando a análise de interoperabilidade das tecnologias HTML5, CSS3, PHP 8 e SQL (MySQL). Aborda de forma aprofundada os mecanismos de autenticação criptográfica, o consumo assíncrono de mídia via Fetch API, segurança contra vetores de ataque XSS e SQL Injection, bem como a refatoração do módulo administrativo sob o padrão arquitetural MVC apoiado na Programação Orientada a Objetos (POO).

Sumário

1. Introdução e Contextualização
2. Análise Profunda do Ecossistema Tecnológico
 - 2.1. HTML5 (HyperText Markup Language)
 - 2.2. CSS3 (Cascading Style Sheets)
 - 2.3. PHP 8 (Hypertext Preprocessor)
 - 2.4. SQL / MySQL (Structured Query Language)
3. Fluxograma Conceitual de Interoperabilidade
4. Arquitetura do Banco de Dados
5. Mecanismo de Autenticação e Registro
6. Provisionamento de Contingência e Semeadura
7. Camada de Visualização e Integração YouTube
8. Módulo Administrativo Avançado (MVC/POO)
9. Encerramento de Ciclo (logout.php)
10. Considerações Finais

Documentação Técnica: Plataforma de Aprendizado DevAcademy

Arquitetura de Sistemas Web, Fundamentação Teórica e Segurança de Dados em Camadas

1. Introdução e Contextualização

O sistema proposto, denominado **DevAcademy**, consiste em uma plataforma de gerenciamento de aprendizado (LMS - *Learning Management System*) simplificada, desenvolvida com o objetivo de demonstrar o isolamento de escopo, controle de sessões e persistência de dados utilizando a infraestrutura fundamental do desenvolvimento web dinâmico: **HTML5**, **CSS3**, **PHP 8** e **SQL (MySQL)**.

O ecossistema visual foi projetado sob uma identidade cromática baseada em variações da cor **roxa**, transmitindo um ambiente de modernidade, foco e criatividade tecnológica. O sistema opera sob uma premissa rígida de **Acesso Restrito Autenticado**, impossibilitando que usuários não validados consumam os recursos internos.

2. Análise Profunda do Ecossistema Tecnológico

2.1. HTML5 (HyperText Markup Language)

- **Definição e Propósito:** O HTML5 fornece a *estrutura semântica* e a arquitetura de informação de um documento web. Ele define o que é um cabeçalho (`<header>`), o que é o conteúdo principal (`<main>`) e o que é um formulário de dados (`<form>`).
- **Importância:** Sem o HTML, a web seria apenas um emaranhado de dados brutos sem nexo visual ou hierárquico para o navegador. O HTML5 introduziu tags semânticas estruturais que melhoram drasticamente a acessibilidade (para leitores de tela usados por deficientes visuais) e a indexação em motores de busca (SEO).
- **Interoperabilidade:** O HTML atua como a **espinha dorsal** ou o "esqueleto" do projeto. Ele expõe uma árvore de elementos conhecida como **DOM (Document Object Model)**. É através do mapeamento dessa árvore DOM que o CSS consegue localizar onde aplicar as cores e o PHP consegue injetar os dados processados dentro das tags corretas.

2.2. CSS3 (Cascading Style Sheets)

- **Definição e Propósito:** O CSS3 é uma linguagem de **folhas de estilo em cascata**. Sua finalidade exclusiva é a *separação de conceitos* (Separation of Concerns), isolando totalmente a estrutura (HTML) da camada de apresentação visual (design, layouts, tipografia, cores e responsividade).
- **Importância:** O CSS transforma um documento estrutural árido em uma interface de usuário (UI) atraente e intuitiva. Ele gerencia a psicologia das cores (como o uso do roxo no projeto para gerar foco) e dita a **responsividade** — a capacidade matemática do layout de se recalcular via *Media Queries* para funcionar perfeitamente em qualquer tamanho de tela.
- **Interoperabilidade:** O CSS trabalha em simbiose direta com o HTML. Ele "escuta" as classes e identificadores (IDs) declarados nas tags HTML e aplica as regras de estilização em cascata. Ele não altera os dados ou a lógica do PHP, mas garante que os dados trazidos pelo PHP sejam exibidos de forma organizada na tela.

2.3. PHP 8 (Hypertext Preprocessor)

- **Definição e Propósito:** O PHP é uma linguagem de **programação interpretada com foco em script do lado do servidor (Server-Side)**. Diferente do HTML and CSS, que rodam no navegador do cliente, o PHP é executado inteiramente no servidor de hospedagem. Sua função é processar a lógica de negócios, tomar decisões, proteger rotas e gerar HTML dinâmico em tempo real.
- **Importância:** O PHP confere inteligência e dinamismo à aplicação. Sem ele, um site seria estático — ou seja, para adicionar uma aula nova, o programador teria que abrir o código fonte e digitar o HTML manualmente toda vez. Com o PHP, o site se torna um sistema maleável, capaz de se adaptar com base em quem está logado e no que existe no banco de dados.
- **Interoperabilidade:** O PHP funciona como o **maestro central** do sistema. Ele faz a ponte entre o Banco de Dados e a interface do usuário. Ele se conecta ao MySQL, extrai os dados brutos, aplica regras de

segurança (como criptografia de senhas), estrutura essa informação e a renderiza para o navegador do cliente.

2.4. SQL / MySQL (Structured Query Language)

- **Definição e Propósito:** O SQL é a linguagem padrão internacional para gerenciamento e manipulação de **Bancos de Dados Relacionais**. O MySQL, por sua vez, é o Sistema Gerenciador (SGBD) que interpreta esses comandos SQL. Serve para armazenar, organizar e recuperar informações de forma estruturada, performática e permanente.
- **Importância:** Em aplicações profissionais, os dados não podem ser salvos em arquivos de texto comuns, pois isso geraria lentidão e corrupção de dados sob acessos simultâneos. O SQL garante a **integridade referencial** (as tabelas se conversam por meio de chaves relacionais) e a consistência, permitindo buscas complexas em milissegundos.
- **Interoperabilidade:** O banco de dados opera de forma isolada na infraestrutura. Ele não sabe o que é HTML ou CSS. A sua única via de comunicação com o ecossistema é respondendo às requisições estruturadas que o **PHP** faz a ele. O PHP envia uma query em texto (Ex: *"Me dê os dados do usuário X"*) e o MySQL responde com as linhas e colunas exatas daquela tabela.

3. Fluxograma Conceitual de Interoperabilidade

1. O **Aluno** digita o e-mail no formulário (**HTML5**), estilizado em roxo (**CSS3**), e clica em Entrar.
2. Os dados viajam até o Servidor. O **PHP** intercepta os dados e dispara uma consulta estruturada (**SQL**) para o **MySQL**.
3. O **MySQL** valida se os dados existem e devolve o registro para o **PHP**.
4. O **PHP** confere a senha, valida a sessão e, se tudo estiver correto, monta uma nova página estruturada em **HTML5**, vincula as regras do **CSS3**, injeta os links dos vídeos obtidos do banco e envia tudo pronto para o navegador do aluno.

4. Arquitetura do Banco de Dados (script.sql e db.php)

A persistência de dados adota o modelo relacional clássico de Entidade-Relacionamento.

- **Tabela usuarios:** Centraliza as credenciais. Destaca-se a coluna `senha`, parametrizada como `VARCHAR(255)` para suportar *hashes* criptográficas expansivas, e a coluna `tipo`, responsável por ditar o nível de privilégio do usuário no sistema (Polimorfismo de Acesso: aluno ou admin).
- **Tabela aulas:** Armazena metadados dos conteúdos, onde o campo `url_video` guarda a referência externa do recurso audiovisual, otimizando o armazenamento do servidor local.

A camada de acesso a dados em `db.php` utiliza a extensão **PDO (PHP Data Objects)**. Academicamente, o PDO é superior às funções procedurais antigas pois implementa o conceito de portabilidade de banco de dados e introduz o uso obrigatório de *Prepared Statements*, mitigando a vulnerabilidade clássica de **SQL Injection**.

5. Mecanismo de Autenticação e Registro (index.php)

A porta de entrada do sistema gerencia o fluxo de controle através de dois estados:

A. Registro (Cadastro)

Ao submeter o formulário de cadastro, o sistema intercepta os dados via método POST. Para garantir a confidencialidade, aplica-se a função nativa `password_hash($senha, PASSWORD_DEFAULT)`, que utiliza o algoritmo **BCrypt**.

Exemplo Prático: Uma senha plana como "123456" é transformada em uma cadeia irreversível de 60 caracteres (ex: `$2y$10$89JbSsn1eCHg...`). Se o banco de dados for comprometido, as senhas continuam seguras.

B. Autenticação (Login)

O PHP efetua uma busca pelo e-mail informado. Caso localize o registro, a função `password_verify($senha, $usuario['senha'])` realiza a checagem criptográfica termo a termo. Sendo válida, o servidor inicializa uma sessão global (`session_start()`) e armazena os dados do usuário na memória do servidor através da superglobal `$_SESSION`.

6. Provisionamento de Contingência e Semeadura de Dados (forcar_admin.php)

Durante a fase de implantação ou manutenção de sistemas, inserções manuais de strings criptografadas diretamente em terminais de bancos de dados frequentemente falham. Isso ocorre devido ao fenômeno de *Mismatch de Codificação de Caracteres* (Unicode/ASCII), onde símbolos estruturais da hash BCrypt (tais como \$, / e .) podem sofrer corrupção de string ou truncamento.

Para solucionar essa limitação de infraestrutura, o sistema incorpora um script de contingência automatizado (`forcar_admin.php`), projetado sob dois conceitos fundamentais de engenharia de software:

A. Idempotência Operacional por Destruição Secura

O script executa uma query destrutiva prévia (`DELETE FROM usuarios WHERE email = ?`) antes de efetuar a inserção. Esse comportamento garante a idempotência do sistema, ou seja, o script pode ser executado múltiplas vezes sem gerar colisões de chave única (UNIQUE KEY) ou duplicidade de administradores no repositório.

B. Consistência Baseada no Ambiente de Execução (Runtime)

Em vez de depender de uma string estática gerada externamente, o script delega a geração do vetor criptográfico para a própria instância do interpretador local do PHP usando:

```
password_hash('admin123', PASSWORD_DEFAULT);
```

Assegura-se compatibilidade absoluta entre o algoritmo gerador e a função de verificação (`password_verify`) utilizada pelo formulário de login no mesmo servidor.

Nota de Segurança de Vetor de Ataque: O arquivo `forcar_admin.php` expõe uma brecha de elevação de privilégios se mantido no servidor de produção. Academicamente, ele é classificado como uma rotina de *semeadura efêmera* (Data Seeding), devendo ser sumariamente excluído do diretório após cumprir sua execução inicial.

7. Camada de Visualização, Consumo de Mídia e Integração com YouTube (cursos.php)

A página de cursos representa a "Área de Membros" dos estudantes, implementando regras estritas de renderização visual e processamento de dados audiovisuais.

A. Validação de Sessão (Gatekeeper)

O rigor de segurança é aplicado na primeira linha de execução do servidor. Se um cliente tentar burlar a interface acessando diretamente a URL do arquivo sem possuir um identificador válido gravado na memória volátil do servidor (`$_SESSION['usuario_id']`), ele sofrerá uma ejeção imediata através do cabeçalho de redirecionamento `header("Location: index.php")`.

B. Engenharia de Strings via Expressão Regular (Regex)

O YouTube bloqueia por padrão a reprodução de seus vídeos em telas externas utilizando URLs convencionais de compartilhamento (como `watch?v=` ou `youtu.be/`), disparando um erro de restrição de cabeçalho *X-Frame-Options*. Para contornar essa barreira de infraestrutura, `cursos.php` implementa um mecanismo de tradução baseado em expressões regulares:

```
$padrao = '/(?:youtube\.com\/(?:[^\w]+\w\/.+\/|(?:(v|e(?:mbed)?))\w\/|.*[?&]v=)|youtu\.be\/|
([^\w?\/ ]{11}))/i';
```

Esta instrução realiza uma varredura léxica na string inserida pelo administrador, isola o identificador alfanumérico único de 11 caracteres que o YouTube atribui ao vídeo e reconstrói o link sob o protocolo de incorporação oficial (`/embed/`). Caso a URL fornecida seja corrompida ou inválida, o sistema intercepta a falha e exibe um tratamento de erro nativo no HTML, prevenindo a quebra da tela do usuário.

C. Layout de Duas Colunas e Responsividade Proporcional

A interface divide o espaço de trabalho (*workspace*) por meio de uma arquitetura limpa: à esquerda, o reprodutor de vídeo central; à direita, a barra lateral de navegação com indexação numérica das aulas (ex: [01], [02]).

Para garantir a reprodução de mídia sem distorções em diferentes resoluções, o contêiner do vídeo utiliza a técnica matemática de estiramento proporcional CSS:

```
.video-container {
    position: relative;
    padding-bottom: 56.25%; /* Proporção 16:9 (9 dividido por 16) */
    height: 0;
}
```

Isso força a tag `<iframe>` incorporada a se redimensionar dinamicamente mantendo a proporção nativa de cinema digital, evitando achatamento de imagem ou faixas pretas horizontais.

8. Módulo Administrativo Avançado: Refatoração MVC, POO e Interatividade Assíncrona (admin.php)

O arquivo `admin.php` passou por uma completa reengenharia de software, abandonando o paradigma procedural de arquivos monolíticos e adotando o padrão arquitetural **MVC (Model-View-Controller)** com **Programação Orientada a Objetos (POO)** e comunicações baseadas na **Fetch API** do JavaScript.

A. Separação de Responsabilidades (Single-File MVC)

- 1. Model (AulaModel):** Classe isolada que encapsula todas as regras de banco de dados e comunicação SQL com o SGBD. Ela recebe a conexão PDO externa via **Injeção de Dependência** no seu construtor, abstraindo operações de leitura (`listarTodas`), gravação (`inserir`) e remoção (`excluir`).
- 2. Controller (AdminController):** Atua como o cérebro lógico do sistema. É responsável por verificar os privilégios de acesso do administrador, interceptar as requisições globais HTTP, validar as strings recebidas e decidir se a resposta enviada ao cliente será uma estrutura de dados **JSON** ou a renderização da interface gráfica.
- 3. View (Camada Visual):** Localizada na base do arquivo, é uma estrutura limpa em HTML5/CSS3 desprovida de qualquer acoplamento com comandos SQL, encarregada apenas de ler o array fornecido pela Controller e desenhar os cards na tela.

B. CRUD Assíncrono com Fetch API e Manipulação do DOM

- **Inserção Assíncrona:** O envio do formulário de cadastro é capturado pelo JavaScript através do método `preventDefault()`. Os dados brutos são serializados em um objeto `FormData` e despachados em segundo plano via `fetch()` para o controlador PHP. O controlador processa, aciona a Model e devolve um cabeçalho `application/json` indicando o sucesso. O JavaScript intercepta a resposta positiva e usa métodos de manipulação de nós do DOM (`insertBefore`) para injetar a nova linha no topo da lista instantaneamente.
- **Deleção Assíncrona por Delegação de Eventos (Event Delegation):** Como novos elementos de aulas são inseridos dinamicamente na tela sem recarregar a página, os botões de exclusão utilizam a técnica de escuta centralizada no contêiner pai (`lista-aulas-container`). O JavaScript identifica o clique no elemento alvo, dispara uma requisição GET assíncrona contendo o ID da aula e remove a linha da tela aplicando um efeito suave de desvanecimento de opacidade (`opacity: 0`).

C. Sistema de Notificações Efêmeras (Toasts) e Segurança Anti-XSS

Para fornecer respostas visuais imediatas das operações do sistema de forma elegante, foi desenvolvido um ecossistema nativo de mensagens denominado *Toasts*. A função JavaScript `dispararToast(mensagem, tipo)` cria dinamicamente caixas estilizadas em roxo (ou vermelho para erros) fixadas no canto inferior direito. Essas mensagens possuem um ciclo de vida efêmero (TTL - *Time to Live*) controlado por temporizadores (`setTimeout`), desvanecendo e se autodestruindo da árvore do DOM após **3 segundos** para evitar vazamento de memória no navegador.

Adicionalmente, para blindar o front-end contra ataques de **XSS (Cross-Site Scripting)** decorrentes da inserção dinâmica de dados, o sistema integra uma rotina de sanitização léxica no JavaScript:

```
function escapeHTML(string) {  
    return string.replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");  
}
```

Esta função intercepta caracteres especiais estruturais e os converte em entidades HTML seguras, impedindo que administradores mal-intencionados injetem tags `<script>` nos campos de cadastro de aulas.

9. Encerramento de Ciclo (logout.php)

O encerramento seguro do ciclo de vida da navegação é gerenciado pelo arquivo de desautenticação. Ele executa sequencialmente três comandos críticos:

1. `session_start()` : Localiza a sessão atual do vetor de requisição.
2. `session_unset()` : Remove todas as variáveis gravadas na sessão.
3. `session_destroy()` : Destrói completamente o arquivo físico de sessão temporária alocado no servidor de hospedagem.

Isso garante que o botão "Sair" invalide permanentemente o token de acesso da máquina cliente, protegendo o usuário em terminais compartilhados.

10. Considerações Finais

A plataforma **DevAcademy** demonstra como boas práticas de engenharia de software podem ser aplicadas desde o nível básico. Ao integrar conceitos modernos como a arquitetura MVC, orientação a objetos, comunicações assíncronas assentes na Fetch API, sanitização proativa de dados no front-end and tratamento inteligente de streams do YouTube, o sistema se consolida como um modelo didático, performático e seguro, totalmente alinhado com as demandas de desenvolvimento web escalável.